

Collaborative Code Editor — Complete Phase-by-Phase Build Guide

Goal: A real-time, multi-user code editor with proper CRDT-based conflict resolution, live cursors, chat, and persistence — fully runnable on your own machine, all free tools.

Project Description (for your README/resume)

A web-based collaborative code editor where multiple users can join a shared coding "room," edit the same file simultaneously, see each other's live cursors, and discuss changes via built-in chat — similar to VS Code Live Share or Google Docs, but for code. Conflict resolution is handled using CRDTs (Conflict-free Replicated Data Types) rather than naive "last write wins," ensuring no user's edits are ever silently lost, even during simultaneous edits to the same line.

Before You Start: Setup

- **Node.js** (v18+) and **npm** installed
 - **VS Code** or any editor for writing your own project's code
 - **MongoDB Community Edition** (free, install locally) or **PostgreSQL** (free, install locally) — pick one
 - **Docker Desktop** (free) — for later deployment phase
 - Basic familiarity with JavaScript/TypeScript, since this stack is all JS-based
-

Phase 1: Basic Code Editor, No Collaboration (Week 1)

Goal: A working code editor in the browser — no real-time features yet.

Steps:

1. Create a React app (`npm create vite@latest my-editor -- --template react`).
2. Install Monaco Editor: `npm install @monaco-editor/react`.
3. Drop it into your app:

```
import Editor from "@monaco-  
editor/react";  
<Editor height="90vh"  
defaultLanguage="javascript"  
defaultValue="// start coding" />
```

4. Confirm you have a working, VS Code–like editor running in your browser with syntax highlighting.

Milestone:  You have a solo code editor working locally.

Phase 2: Real-Time Sync Between Two Browser Tabs (Week 2)

Goal: Open the same "room" in two browser tabs and watch text sync live between them.

Steps:

1. Install Yjs and its Monaco binding: `npm install yjs y-monaco y-websocket`.
2. Run the free reference WebSocket server (comes bundled with `y-websocket`) locally: `npx y-websocket-server` (runs on `localhost:1234` by default).
3. In your React app, create a Yjs document and connect it to your Monaco editor:

```
import * as Y from "yjs";
import { WebSocketProvider } from "y-
websocket";
import { MonacoBinding } from "y-
monaco";

const ydoc = new Y.Doc();
const provider = new
WebSocketProvider("ws://localhost:1234"
, "room-name", ydoc);
const yText = ydoc.getText("monaco");
// then bind yText to your Monaco
editor instance using MonacoBinding
```

4. Open two browser tabs pointed at the same room name, type in one, watch it appear in the other.

Milestone:  Real-time sync working — this is your biggest technical milestone, don't rush past it.

Phase 3: Understand and Verify Conflict Resolution (Week 3)

Goal: Prove your editor correctly handles simultaneous edits — this is the part that makes your project stand out.

Steps:

1. Deliberately test the hard case: open two tabs, position both cursors on the exact same character, and type in both at the same instant (or use two people on two machines).
2. Confirm both edits appear correctly in both tabs, in the same final order, with nothing deleted or duplicated.
3. Read the Yjs documentation section explaining *why* this works (unique IDs per character, relative positioning) — write a short note in your own README explaining it in your own words. This becomes your interview-ready explanation.
4. (Optional, for extra depth) Try building a tiny standalone script — no editor, just plain Yjs — that simulates two "users" inserting characters out of order, and print the final merged result, to see the CRDT logic in isolation.

Milestone:  You've verified and can explain the core conflict-resolution mechanism — this is what separates your project from a toy tutorial.

Phase 4: Backend Server, Rooms, and Basic Auth (Week 4)

Goal: Move from "everyone uses the same hardcoded room" to real user accounts and multiple named

projects/rooms.

Steps:

1. Set up a Node.js + Express server (`npm install express`).
2. Build basic auth:
 - `POST /signup` and `POST /login` routes.
 - Store users (username, hashed password) in MongoDB/PostgreSQL.
 - Use `jsonwebtoken` to issue a JWT token on login; require it on protected routes.
3. Build room management:
 - `POST /rooms` → create a new room (project), save to database with an owner and a unique room ID.
 - `GET /rooms` → list rooms a user has access to.
 - `GET /rooms/:id` → join a specific room.
4. On the frontend, use the room ID as the Yjs "room name" when connecting to the WebSocket provider, so each room has its own independent document.

Milestone:  Multiple real users can create and join separate, isolated coding rooms.

Phase 5: Live Cursors and Presence (Week 5)

Goal: See who else is in the room and where their cursor is — this is what makes it feel truly "collaborative," not just synced.

Steps:

1. Yjs has a built-in feature for this called **Awareness** — it's designed exactly for sharing temporary info like cursor position and username (as opposed to the permanent document content).
2. Set a local user's awareness state (name, a random color, cursor position) whenever they move their cursor in Monaco.
3. Listen for awareness changes from other users and render their cursor position and name as an overlay in the editor (Monaco supports custom decorations for this).

Milestone:  You can see other users' names and cursors moving live, like real VS Code Live Share.

Phase 6: Chat / Discussion Feature (Week 6)

Goal: Let users discuss changes without leaving the editor.

Steps:

1. Add a simple chat panel component in React, alongside the editor.
2. Use Socket.IO (`npm install socket.io socket.io-client`) for a separate real-time channel dedicated to chat messages, scoped to the room ID.
3. On the server, relay chat messages to everyone in that room's socket channel.
4. Save chat messages to your database (linked to the room ID) so history persists when users rejoin.

Milestone:  Users can code and chat together in the same interface.

Phase 7: Persistence for the Code Itself (Week 7)

Goal: Don't lose the actual code if everyone leaves and the server restarts.

Steps:

1. Periodically (e.g., every 30 seconds, or on every meaningful change with debouncing) serialize the

current Yjs document state and save it to your database, tied to the room ID.

2. On room creation/join, check if saved state exists in the database — if so, load it into a fresh Yjs document before anyone starts editing.
3. (Optional, cleaner approach) Look into `y-mongodb` or similar community persistence adapters built specifically for Yjs, which handle this pattern for you.

Milestone:  Code is never lost, even across server restarts.

Phase 8: Deployment — Docker, All on Your Machine (Week 8)

Goal: Package everything so it runs with a single command, no cloud required.

Steps:

1. Write a `Dockerfile` for your frontend (React build) and one for your backend (Node.js server).
2. Write a `docker-compose.yml` that spins up: frontend, backend, MongoDB/PostgreSQL, and the y-websocket server, all networked together.
3. Run everything with one command: `docker-compose up`.

4. Test the full flow end-to-end: sign up, create a room, invite a "second user" (open an incognito tab), edit together, chat, close everything, restart, confirm your code and chat history are still there.
5. **Optional:** deploy to a free tier on Render.com or Railway.app if you want a live public link for your resume, in addition to the local Docker setup.

Milestone:  A fully working, deployable, resume-ready project.

Stretch Goals (if you have extra time)

- **Run code button** — integrate a free sandboxed code execution API (Judge0 has a free tier) so users can actually run the shared code together.
 - **Version history** — Yjs tracks all changes internally, so you can build a simple "time travel" slider to view the document's history.
 - **Syntax-aware features** — leverage Monaco's built-in support for multiple languages, linting, and autocomplete.
 - **Voice chat** — using free WebRTC libraries, for an extra layer of real-time collaboration.
-

Suggested Project Structure

```
collab-code-editor/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Editor.jsx          //
│   │   │   └── Monaco + Yjs binding
│   │   │       ├── Chat.jsx        //
│   │   │       └── Socket.IO chat panel
│   │   │           ├── Cursors.jsx  //
│   │   │           └── Awareness-based live cursors
│   │   │               └── RoomList.jsx //
│   └── create/join rooms UI
│       ├── App.jsx
│       └── Dockerfile
├── backend/
│   ├── src/
│   │   └── server.js                //
│   └── Express + Socket.IO + y-websocket setup
│       ├── routes/
│       │   ├── auth.js
│       │   └── rooms.js
│       ├── models/
│       │   ├── User.js
│       │   ├── Room.js
│       │   └── ChatMessage.js
│       ├── db.js
│       └── Dockerfile
├── docker-compose.yml
└── README.md
```

Resume Bullet (once complete)

Built a real-time collaborative code editor supporting concurrent multi-user editing using CRDT-based conflict resolution (Yjs), live cursor presence via the Awareness protocol, integrated room-scoped chat, and persistent storage; fully containerized with Docker for one-command local deployment.

If You Get Stuck

- The Yjs official docs (yjs.dev) have a dedicated, beginner-friendly section explaining CRDTs and walking through exactly this kind of editor integration.
- The `y-monaco` and `y-websocket` GitHub repos both include working example projects you can reference (read to understand the pattern, then build your own version with your own room/auth/chat logic on top).
- If real-time debugging gets confusing, add simple console logs on both the client and server showing what "operations" are being sent/received — this makes the CRDT sync visible and much easier to reason about.

Take it phase by phase — even stopping after Phase 5 (live cursors) gives you a genuinely impressive, demo-able project.